Linkwise: Radio, Modem, and Link Management Research

Lab notes and evidence ledger

Engineering research log

10 November 2026

Abstract

This log records the initial setup and reconnaissance for an independently implemented Rust modem intended to interoperate with VARA HF and be installable on macOS, Windows, and Linux. It distinguishes direct observations from hypotheses and local tests from reference-peer qualification. Native BW500 codec, streaming receive, audio-handshake, DATA/ARQ, and host-stream components now exist. The 9 November entry records their implementation and validation. The 10 November entry adds a shared session runtime, a clock-driven runner, and a two-way BW500 audio loopback. Operational interoperability remains open.

Contents

1 Project objective and boundaries **3**
- 1.1 Objective . . . 3
- 1.2 Research boundaries . . . 3

2 Chronicle **3**

3 Reference materials and provenance **12**
- 3.1 Supplied VARA package . . . 12
- 3.2 Documentation age . . . 12

4 Ghidra setup and analysis ledger **13**
- 4.1 Setup . . . 13
- 4.2 Imported programs . . . 13
- 4.3 Evidence ledger . . . 13
- 4.4 Saved Recon bookmarks . . . 14

5 Rust scaffold and cross-platform environment **14**

6 Workspace organization **15**

7 Implementation roadmap (proposed) **15**
- 7.1 Planning targets and architecture . . . 16
- 7.2 Phases and exit gates . . . 16
- 7.3 Cross-cutting standards and acceptance . . . 17
- 7.4 Initial execution order . . . 18
- 7.5 Progress against the roadmap (2026-11-07) . . . 18

8 Signal data acquisition and corpus (proposed) 19
8.1 Purpose and evidence policy . . . 19
8.2 Candidate data sources . . . 19
8.3 Recording preservation and metadata . . . 20
8.4 Coverage and capture campaigns . . . 21
8.5 Storage, analysis, and corpus admission . . . 21
8.6 Initial acquisition sprint and gate . . . 22

9 Roadmap unblock plan 22
9.1 Primary blocker and objective . . . 22
9.2 Ordered recovery and decision gates . . . 22
9.3 Stop conditions and escalation . . . 23

10 Readiness snapshot: 2026-11-07 (historical) 24

11 2026-11-09: Native modem implementation and verification 24
11.1 Implementation changes . . . 25
11.2 Evidence provenance and reproducible checks . . . 26
11.3 Attempted reference runtime: failed environment gate . . . 26
11.4 Stage status and unresolved gates . . . 27
11.5 Whitepaper build and overflow fixes . . . 27

12 2026-11-10: Shared runtime and BW500 audio loopback 27
12.1 Architecture and implementation . . . 27
12.2 Simulated audio experiment . . . 28
12.3 Reproduction and results . . . 28

13 2026-11-10: KiwiSDR receive evidence and pilot acquisition 29
13.1 Host and platform work . . . 29
13.2 Live capture and first verified frame . . . 29
13.3 Acquisition change and verification . . . 29

14 Chronicle maintenance 30

15 Next work 30

16 References 31

1 Project objective and boundaries

1.1 Objective

Develop an independent, cross-platform Rust modem that can interoperate with VARA HF. Intended initial operating-system families are macOS, Windows, and Linux, with ARM64 and x86_64 build targets for each. Hardware and native audio validation will be required; compile checks on one host do not demonstrate device-level compatibility.

1.2 Research boundaries

- Use the supplied reference application and controlled observations to infer externally observable behavior. Keep the Rust implementation independent; do not copy decompiled implementation code or proprietary assets into the Rust crate.
- Keep confirmed facts, hypotheses, and unresolved questions separate. A UI string or function cross-reference is a lead, not proof of protocol semantics.
- Review applicable licenses and law before analysis or redistribution.
- Limit dynamic transmission tests to audio loopback or a properly shielded/conducted test setup; avoid unintended over-the-air transmissions.

2 Chronicle

Date	Work recorded
2026-10-30	Reviewed the product description and local resources. Established the goal as an independent Rust reimplementation with macOS, Windows, and Linux support. The vendor description identifies computer sound-card operation, adaptive link management, ARQ, and a TCP command interface as product-level features; these descriptions are not a detailed protocol specification.
2026-10-30	Installed Ghidra 12.1.4 alongside the existing 12.2 user profile because the provided GhidraMCP 7.0.0 extension declares Ghidra 12.1.4 compatibility. Installed the extension in the 12.1.4 profile, used Java 21, and registered the workspace bridge in `.vscode/mcp.json`. The MCP health endpoint reported version 7.0.0 at port 8089.
2026-10-30	Set up the VARA source artifacts and Ghidra analysis project. Imported the installer and then the extracted application executable. Added Recon bookmarks to the application analysis and saved the Ghidra project.
2026-10-30	Installed UTM 5.0.8 and CrystalFetch 2.2.0 to prepare a Windows 11 ARM64 reference VM on an Apple Silicon Mac. The user reported on 2026-10-30 that the Windows 11 ISO was still downloading. At the last available workspace check, no ISO was visible in the accessible Downloads directory. The Documents directory was inaccessible to the tool session, and UTM CLI automation returned macOS Apple Events permission error -1743. VM creation and reference execution remain pending.

Date	Work recorded (continued)
2026-10-30	Installed Rust stable 1.99.1, Cargo, rustfmt, Clippy, and rust-analyzer. Added target standard libraries for macOS ARM64/x86_64, Windows ARM64/x86_64, and Linux ARM64/x86_64. All six targets passed a `cargo check` of the initial dependency-free crate.
2026-10-30	Created the Rust scaffold in `rust/`, with separate API, audio, portable modem, and platform modules. Formatting, Clippy, tests, and CLI `-help`/`-version` smoke checks passed. There are currently zero implemented tests and no modem behavior.
2026-10-30	Organized supplied reference materials under `reference/vara/` and `reference/ghidra/`, preserving the active Ghidra project, `.venv/`, and VS Code MCP configuration at their existing paths. Added workspace overview documentation. The workspace is not a Git repository, so no Git status/diff is available.
2026-10-30	Created this LaTeX lab log and a Beamer progress deck. Recorded the verified setup, reference fingerprints, initial static-analysis clues, Rust scaffold checks, and pending Windows VM work. Going forward, update the chronicle as work proceeds, noting material actions, evidence, validation results, and blockers before concluding each work session.
2026-10-31	Located the downloaded Windows installer in Downloads: `26100.4349.250607-1500.ge_release_svc_refresh_CLIENTCONSUMER_RET_A64FRE_en-us.iso`. The file is a bootable ISO 9660 image, 5,267,005,440 bytes. Its filename identifies an AArch64 build, but the Windows image architecture has not yet been verified from its install image. Local SHA-256: `347624272ec7e209de37ff5a5ff49190a616eec792e176fd4ba19fd78f278e2a`. This checksum was measured locally; it has not been compared against a publisher-provided checksum. UTM 5.0.8 is installed and open, but `utmctl` still receives macOS Apple Events permission error -1743. No UTM virtual machine has been created.
2026-10-31	Located an open-source UTM MCP server and registered it alongside Ghidra in `.vscode/mcp.json`. The published `mcp-utm==0.1.2` package failed to start because its dependency resolver installed MCP SDK 2.x while its code imports the removed `mcp.server.fastmcp` API. Switched to the MIT-licensed `neverprepared/mcp-utm` source at pinned commit `5f6bab58061f995b90cd5a5af4101211b6768fd4`, constrained the MCP SDK to version 1.x, and verified a real stdio handshake, enumeration of all 22 registered tools, and a successful `list_vms` call. The inventory is empty; no VM was created. The server exposes state-changing tools, including permanent VM deletion; no such action was invoked.
2026-10-31	The user created a UTM VM named `Windows`. Via the UTM MCP, verified that it is registered with the QEMU backend, has 8,192 MiB memory, and has a 5,031 MiB removable drive attached (consistent in size with the downloaded Windows ISO). Started the VM and confirmed its status is `started`. The MCP configuration reports `cpu_cores=0`; this may be an unsupported/unknown QEMU configuration value and must be checked in UTM before relying on a CPU allocation. Windows setup has not yet been observed or completed.

Date	Work recorded (continued)
2026-10-31	The user reports Windows installation is complete, the guest desktop was reached, and ARM64 architecture was confirmed. This is user-reported status; the MCP does not provide direct guest-screen evidence. The VM inventory and configuration were re-queried: `Windows` is registered as QEMU and running, with 8,192 MiB memory and three drives, including a removable 5,031 MiB drive. The MCP still reports `cpu_cores=0`, which should be checked in UTM before running the reference modem. Next: verify guest readiness and CPU allocation, then install and test VARA HF v4.9.0.
2026-10-31	The user checked UTM and reports that the VM is configured with 10 virtual processors. This resolves the CPU-allocation check; the earlier MCP value of `cpu_cores=0` was not an accurate indication of the configured count. The VM is ready for the next gate: install and test VARA HF v4.9.0.
2026-10-31	Before guest installation, reviewed the extracted package's `License.rtf`. It describes VARA HF as shareware and includes a term prohibiting reverse compilation, modification, translation, and disassembly. That restriction conflicts with this project's VARA reverse-engineering goal. No guest installation or dynamic behavioral testing was performed during this license review.
2026-10-31	The user states that they have written authorization covering the entire VARA reverse-engineering and testing scope. This is recorded as a user-reported authorization; the authorization document was not reviewed or retained. Work may proceed within the user's stated scope. The UTM VM has no shared folders, and the available UTM MCP does not expose a guest shell or guest UI-control tool. Prepared a separate 4.4 MiB ISO containing only the provided VARA HF v4.9.0 setup executable for transfer into the guest. The original installer remains unchanged. Attaching the ISO requires a stopped VM; the user authorized a graceful stop after saving guest work.
2026-10-31	Gracefully stopped and restarted the Windows VM with user approval. The installer ISO was validated locally, but UTM MCP's `attach_drive` operation failed with AppleScript error -10006; the VM was not left stopped. The installed `utmctl` supports guest file push/exec, but the Windows guest reports that the QEMU guest agent is not installed/running. Verified the UTM shared-network host address is 192.168.64.1. Continue with a temporary host-local transfer of the single setup executable; the user must complete Windows installer UI and license prompts because no guest UI-control channel is available.
2026-10-31	Staged the original VARA HF v4.9.0 setup executable in a temporary directory and exposed only that file through a temporary HTTP endpoint bound to the UTM private interface at 192.168.64.1:8765. A host-side download smoke test matched the source SHA-256 `b7d67fd9fcc2043a49e58ae8202fc699bdb6520255d21aefd258260c767e04bd`. No installer was run in the guest. The endpoint is temporary and will be stopped after the user confirms the download.

Date	Work recorded (continued)
2026-10-31	The user reports that VARA is installed in the Windows VM and identifies `~/Downloads/utm-share/` as the UTM shared folder. The UTM MCP now confirms that directory is registered for the running VM. Its `app/` payload was compared recursively with the extracted v4.9.0 reference payload; no differences were found. The user's installation report has not yet been independently verified inside the guest, and no modem runtime test has been performed. Stopped the temporary HTTP transfer server after this update; the shared folder is now the intended host/guest data path.
2026-10-31	Continued static command-interface reconnaissance in the existing Ghidra program. The UI string `TCPCommandPort(default:8300)` at `00448260` has a data reference in `FUN_004c3780`; invalid command/KISS TCP port messages reference `FUN_004c61c0`. The strings `CQFRAME`, `LISTENCQ`, `LISTENON/OFF`, `DRIVELEVEL`, `VERSION`, `CONNECTED`, `DISCONNECTED`, `BUSYON/OFF`, and `BITRATE` are referenced by code including `FUN_004cdf80` and nearby event handlers. A UI description says the KISS interface is for APRS applications and should be disabled when running Winlink. These are string/reference observations only; they do not establish command framing, direction, responses, or behavior. Decompilation attempts for the implicated functions again returned `cancelled`; assembly context confirms string constants are used at those code locations but is not sufficient to specify the protocol. Runtime validation remains pending. The UTM VM is running and the shared directory is registered, but its guest IP was not discoverable from the host's current ARP table and the QEMU guest agent is unavailable. No packets or commands were sent to the guest.
2026-10-31	The user reports that Windows has no network device. UTM MCP reports the VM network mode as `shared`; however, no guest IPv4 address is available from `utmctl` and the host ARP table has no guest entry. UTM's Windows guest guide says SPICE guest tools include the network drivers and that networking may not work until they are installed. Recommended next step is to shut down the guest cleanly, confirm networking is enabled in the VM's QEMU settings with Shared Network mode, install UTM's Windows Guest Tools from its CD menu, reboot, and check Windows Device Manager and `ipconfig`. UTM/libslirp may not support ping, so lack of ping replies alone is not a failure signal.
2026-10-31	Downloaded the official UTM Guest Tools ISO from `https://getutm.app/downloads/utm-guest-tools-latest.iso` to `~/Downloads/utm-share/utm-guest-tools-latest.iso`, the user's UTM shared folder. The 121 MiB image is ISO 9660 and contains `utm-guest-tools-0.1.271.exe` and the VirtIO Windows driver license text. SHA-256: `65b6a69b392ee01dd314c10f3dad9ebbf9c4160be43f5f0dd6bb715944d9095b`. UTM's official Windows guest-tools guide states Windows ARM64 is supported. This is the appropriate UTM/QEMU guest driver bundle; VMware VMXNET3 is not the adapter model configured for this VM. The ISO has not yet been mounted in the VM or installed.

Date	Work recorded (continued)
2026-10-31	The user clarifies that Windows reboots when the VM starts with either VirtIO PCI or Intel E1000 configured; no stop code or other error is reported. The UTM process itself is not reported as crashing. UTM still reports the VM as started, but no guest IPv4 address is discoverable. Next diagnostic step: fully stop the VM, disable/remove the emulated network device temporarily, and test whether Windows boots stably. If it does, install UTM Guest Tools from the attached Guest Tools CD while networking is disabled, then re-enable a UTM-compatible adapter and retest. Change only one variable at a time; do not reconfigure CPU or additional devices during this isolation check.
2026-10-31	The user asks whether Wine could be used as an alternate VARA research environment. Assessment: feasible as a compatibility experiment, but not a replacement for Windows validation. The imported executable is 32-bit x86 and the application package contains VB6 ActiveX controls (including Winsock and serial-control components); audio and COM behavior may differ under Wine. No Wine environment has been installed or run. Before proceeding, choose a host/guest architecture that can run this 32-bit application and preserve the authorized scope; compare only non-transmitting behavior first.
2026-10-31	The user proposes a Wine environment inside an x86 Linux VM. This is a reasonable fallback to investigate while the Windows guest has startup reboots: an x86_64 Linux guest can run under QEMU emulation on the Apple Silicon host, and Linux typically has built-in VirtIO networking. Expect lower VM performance than ARM64 virtualization, and treat VARA's VB6 ActiveX/Winsock/audio/serial behavior under Wine as compatibility-specific. No ISO has been downloaded and no Linux VM has been created. Proposed setup: Debian amd64 guest, snapshot before installing Wine, install 32-bit Wine support and required VB6/COM dependencies, then test VARA idle and non- transmitting before attempting any audio work.
2026-10-31	The user selected Fedora for the x86 Linux Wine VM. Downloaded Fedora Workstation 43 Live x86_64 from the official Fedora release directory into `~/Downloads/utm-share/Fedora-Workstation-Live-43-1.6.x86_64.iso`. The bootable ISO is 2,742,190,080 bytes. Its SHA-256 `2a4a16c009244eb5ab2198700eb04103793b62407e8596f30a3e0cc8ac294d77` matches the hash in Fedora's published CHECKSUM file. The CHECKSUM signature was not independently verified. The ISO and CHECKSUM file are now in the shared folder. No Fedora VM has been created; UTM should use Emulate with x86_64 architecture (not ARM virtualization).
2026-10-31	Checked host QEMU availability before attempting the requested QEMU/KVM installation. This Apple Silicon Mac already has Homebrew QEMU 11.1.1 installed. `qemu-system-x86_64 -accel help` lists only TCG; the ARM QEMU binary lists HVF and TCG. KVM is a Linux-kernel accelerator and is not available on macOS. No package installation or upgrade was performed. An x86_64 Fedora guest can run under TCG emulation but will be substantially slower; native ARM64 guests can use HVF.

Date	Work recorded (continued)
2026-10-31	Downloaded Fedora Everything netinst x86_64 43-1.6 to the shared folder. Its SHA-256 `f4d06a40ce4fb4a84705e1a1f01ca1328f95a2120d422ba5f719af5df62d0099` matches Fedora's published CHECKSUM entry; the manifest signature was not independently verified. Added `tools/fedora-qemu-install.sh`, which creates a separate 64 GiB QCOW2 disk and starts a Kickstart-based Workstation install under QEMU TCG. The existing UTM `Linux` VM is left untouched. The automated install was attempted, but QEMU exited without producing a usable Fedora installation. The QCOW2 image is structurally valid with a 64 GiB virtual size but only 196 KiB allocated; no Fedora first boot was verified. The script refuses to overwrite this disk.
2026-10-31	Continued read-only Ghidra analysis without a Windows runtime. Cross-references map `CQFRAME`, `LISTEN ON/OFF`, and `VERSION` to `FUN\_004cdf80`; the listen strings also reference `FUN\_004d45a0`. Status strings reference separate candidates: `CONNECTED` to `FUN\_004d14e0`, `DISCONNECTED` to `FUN\_004d1ac0`, `BUSY ON/OFF` to `FUN\_004d2640/ FUN\_004d27a0`, `BITRATE` to `FUN\_004d2da0`, and `VERSION VARA` to `FUN\_004d33b0`. Decompilation still returns `cancelled`; these are string-reference leads, not verified command handlers or semantics. Both UTM guests remain stopped.
2026-11-01	Drafted an implementation roadmap from the supplied proposal. The phases, schedule, crate boundaries, and candidate waveform parameters are planning assumptions, not measured VARA behavior or completed work. The roadmap keeps protocol and DSP implementation gated on reference evidence and adds the project's existing macOS target to the proposal's Windows/Linux scope.
2026-11-01	Captured a proposed signal-data acquisition guide in this log. It prioritizes authentic VARA HF recordings, immutable originals, provenance, sidecar metadata, hashes, evidence labels, and validated-corpus gates. Its suggested tools, formats, and capture campaigns are recommendations; no new recordings or signal measurements were made by documenting the guide.
2026-11-01	Added an unblock plan to sequence reference-runtime recovery, a first controlled bidirectional capture, corpus validation, and parallel Rust foundation work. The plan is prospective; no VM recovery or signal capture was performed in this update.
2026-11-01	Expanded the OpenVARA architecture whitepaper to describe the AI's channel planning loop: VOACAP/VOACAPL provides timestamped statistical path and frequency/time estimates for eligible Winlink RMS channel records; the ranker combines these with RMS status, station profile, reported/measured conditions, and prior session outcomes. The paper distinguishes candidate channel selection from creating an RMS service or allocating spectrum, and states that propagation estimates do not guarantee a live connection.

Date	Work recorded (continued)
2026-11-01	Added an isolated research path for predicted satellite reentries. KeepTrack v4 documents upcoming reentry predictions sourced from Space-Track TIP messages; these estimates are not confirmed event times or measurements of ionospheric effects. The proposed workflow records prediction provenance and uncertainty, then studies temporal/geographic associations with sporadic-E / ionosonde observations and measured link outcomes. Live ranking excludes this feature by default; paired baseline, reentry-feature, and shuffled-covariate tests with held-out events/paths/seasons are required before an operator may enable a bounded contribution. The cited ION 2026 paper examines a possible association, not a validated HF prediction method.
2026-11-01	Updated the PlantUML system-context diagram to place reentry data behind a shadow-only research/validation gate, and amended the RF simulation design with reentry ablation and negative-control experiments. Rebuilt the LaTeX whitepaper and all three PlantUML figures successfully. Tectonic emitted non-fatal layout warnings (overfull/underfull boxes); the PDF was generated. This is a document/design update only: no VOACAP runs, radio tests, link measurements, or validation experiments were performed.
2026-11-01	Renamed the project from OpenVARA to Linkwise to better represent its full scope: modem interoperability research, radio control, propagation-informed Winlink link management, and AI-assisted connectivity optimization. Updated the maintained documentation, PlantUML sources, Rust package/binary, editor tasks, and whitepaper filename. Earlier chronicle entries retain the project name used when those records were written; the reference material itself was not renamed.
2026-11-07	Reviewed Wing et al.'s lidar study of the 19 February 2025 Falcon 9 uncontrolled reentry and catalogued it in the propagation toolbox. The paper reports an approximately tenfold lithium enhancement near 96 km over Kühlungsborn about 20 hours after reentry; wind-informed back trajectories connect the sampled air mass to the reentry path west of Ireland. Ionosonde, meteor-radar wind, and geomagnetic observations did not support a natural sporadic-E-related explanation for this specific lithium plume. This is direct evidence of a measurable atmospheric metal signature associated with a rocket reentry, not evidence of an HF/10 m opening or improved Winlink link. The reentry/propagation relationship remains a hypothesis requiring repeated event and control periods with independent ionosonde, 10 m, and link-outcome records. Documentation update only; no propagation measurements or Linkwise experiments were performed.

Date	Work recorded (continued)
2026-11-07	Started the Linkwise Rust MCP radio-control implementation. The service exposes typed state, capability, control, and PTT tools over stdio; CAT access uses a bounded `rigctld` adapter with writes disabled by default, station-profile limits, readback checks, and JSONL audit records. Compose profiles cover local development, simulation, and a Hamlib 4.7.2 `rigctld` adapter; the Hamlib source release is pinned by SHA-256. A workspace stdio MCP entry now launches the simulator through Compose; simulator frequency, mode, and power changes exist only in process memory, and PTT key-up remains denied. Both container images built, and MCP state/capability requests succeeded against the simulation backend and Hamlib's dummy backend. An MCP control session changed the simulated frequency to 7.1 MHz, mode to CW, and normalized power to 0.2; readback and revision checks passed, while key-up was denied. Three Rust unit tests passed; the physical radio path has not been validated.
2026-11-07	Reconciled the implementation roadmap with the current code and validation evidence. The containerized radio MCP work is recorded as a partial Phase 7 slice; Phase 0 remains open, Foundation 0.1 and Phases 2–6 remain incomplete, and Phase 8 is only partially established. Added the remaining MCP safety and integration-test gaps to next work. No new VARA protocol behavior or modem interoperability was established by this documentation update.
2026-11-07	Extended the Hamlib capability catalog in `rust/src/radio.rs`. Capability discovery now parses a single `dump_caps` response for get/set function, level, and parameter catalogs; capability flags; mode/VFO enumerations; and numeric ranges. This replaces the previous series of live queries during catalog construction, which caused the MCP capability call to stall against the dummy backend. Common VFO, passband, split VFO, antenna, memory-channel, RIT/XIT, split frequency/mode, and generic Hamlib controls are represented with typed descriptors. Added parser coverage for tab-separated flags and numeric catalog tokens; all seven Rust unit tests pass. Rebuilt the Linkwise container and exercised `radio_get_capabilities` through Compose against Hamlib's dummy model: frequency, mode, VFO, and RFPOWER were reported supported/readable, and physical writes remained disabled by default. Updated the whitepaper and rebuilt its PDF. Memory records, vendor-extension invocation, typed parameter enums, and several non-control rig operations remain incomplete; no physical radio or VARA behavior was tested.

Date	Work recorded (continued)
2026-11-07	Expanded the whitepaper's architecture documentation with six as-built PlantUML views of the current Rust MCP implementation: runtime components, Compose deployment, radio capability/read flow, typed control writes, write validation, and PTT behavior. These views are identified separately from the proposed Linkwise system architecture to avoid implying that proposed session, authorization, or watchdog behavior is implemented. Updated the whitepaper guide to identify PlantUML as the diagram source of truth. All 13 figures are placed on distinct landscape pages with captions; image dimensions are bounded to the available page area while retaining aspect ratio. Rebuilt the whitepaper and visually reviewed the figure pages; diagrams and captions fit without overflow. Tectonic reported non-fatal layout warnings. This was a documentation and presentation change only: no radio, MCP, or VARA implementation behavior changed, and no hardware or modem tests were performed.
2026-11-07	Extended the proposed evidence and decision-history architecture after a consistency review. The evidence model now distinguishes bounded provider collection runs, typed immutable observations, forecast runs, inventory and station-profile revisions, exact analysis-input references, and versioned historical outcome aggregates. The decision-history model records individual ranking entries, select/wait/no-route decisions, policy evaluations, and separate session attempts and lifecycle events for retries. Aligned the software design, system data view, MCP response descriptions, Winlink tracking model, and functional requirements with those records and their provenance, deduplication, replay, stale/partial-collection, and measured-outcome rules. Updated the propagation-flow, software-layers, and information-model diagrams, then rebuilt the whitepaper and regenerated its PDF successfully. The build reports non-fatal LaTeX layout warnings. This is proposed architecture and documentation work only: no collector, analyzer, persistence implementation, radio operation, or propagation validation was performed.
2026-11-07	Reorganized the whitepaper so the communications background, link/session model, propagation foundations, and safety requirements precede architecture. Removed the as-built prototype subsection and its figures from the main paper; implementation status and validation evidence remain in the earlier dated chronicle entries above. PlantUML source files are retained under `docs/whitepaper/diagrams/`. The paper retains a brief maturity note that points to this chronicle. Documentation change only; no code, radio, or modem behavior changed, and no hardware tests were performed.

Date	Work recorded (continued)
2026-11-07	Clarified the RF observation and modem contracts in the whitepaper. The RF adapter now distinguishes SDR spectrum measurements from IC-7200 demodulated audio observations, carries measurement provenance, timing, estimator, uncertainty, and validity, and treats transmit/desense intervals as coverage gaps. The modem contract defines link-up as the modem protocol state machine's completed bidirectional handshake and separates that event from RF activity, Winlink session/transfer completion, and confirmed CMS acceptance or delivery. Rebuilt the whitepaper PDF successfully; LaTeX emitted non-fatal layout warnings. Documentation only; no radio, DSP, modem, or Winlink runtime behavior was implemented or tested.

3 Reference materials and provenance

3.1 Supplied VARA package

The local HF package is the VARA HF v4.9.0 setup ZIP in `reference/vara/installers/`. It contains `VARA setup (Run as Administrator).exe`. The setup executable was identified as a 32-bit Windows PE GUI installer. Its strings identify Inno Setup setup data 5.1.2. The installer payload was extracted using `innoextract` to `reference/vara/extracted-v4.9.0/app/`.

The extracted `VARA.exe` is a PE32 x86 GUI application. Ghidra identified language `x86:LE:32:default`, compiler `windows`, image base `00400000`, 808 functions, and 9,868 symbols. The on-disk executable size is 1,310,720 bytes. These are tool observations about this imported image, not claims about the architecture of unseen components.

Artifact	Size (bytes)	SHA-256
`VARA.exe`	1,310,720	`c69b8c81f888032a67f3c15fe57ef5ccdabef6eba5be148a5a4d7eb44f00486f`
`VARAHF500.dat`	734,771	`4be7fd1a8f1ee2df4c529e8d4c34205fa06a26b82fbceb02abde7295c049ded3`
`VARAHF2300.dat`	720,716	`2c823ce719944dccf90194129b7c95f91b9add7c33a46bb0c80b8401d0edcfe5`
`VARAHF2750.dat`	872,206	`e3e516f70f7710dc7992196573ce0141f5604d978e2701dced26eb06de3aede6`
`Data.dat`	65,000	`58590d0ba95f819f5bccca97291043db96405b8304b244121c702bec6d994c23`

The three `VARAHF*.dat` files are distinct payloads whose names include 500, 2300, and 2750. Their internal roles and formats have not been confirmed. Do not treat filenames or entropy measurements as a DSP specification.

3.2 Documentation age

The extracted application bundle contains a quick guide marked VARA HF 4.7; it is older than the v4.9.0 executable and must be treated as potentially stale. The v4.9.0 ChangeLog and executable

strings are the more version-specific local leads. The product website is useful for high-level context, not implementation detail.

4 Ghidra setup and analysis ledger

4.1 Setup

Ghidra 12.1.4 and extension GhidraMCP 7.0.0 are installed locally. The workspace-level MCP configuration points to the local Python bridge executable. The plugin health check returned status `ok`, server kind `gui`, version `7.0.0`. The project shows 205 plugin endpoints; default tool groups were available.

The active project is represented by `vara.gpr` and `vara.rep/`. The workspace copy of `vara.gpr` is zero bytes in the filesystem snapshot, while the repository directory contains project state and indexed data. Preserve both together and validate the project through Ghidra rather than inferring validity from the stub file alone.

4.2 Imported programs

- Installer loader: `VARA setup (Run as Administrator).exe`, imported under Ghidra project folder `/recon`; Ghidra detected x86 32-bit Delphi and 456 functions.
- Application: extracted `VARA.exe`, imported under project folder `/reference/VARA_HF_4.9.0`; analysis reported 808 functions.

4.3 Evidence ledger

Class	Observation	Interpretation / limit
Verified	The GUI text includes `TCP Command Port (default: 8300)` at `00448260`; it also includes TCP port range validation text.	Confirms a displayed command-port setting/default in this build. Does not establish command grammar, line framing, default runtime bind address, or which socket serves every application function.
Static clue	The bundled VARA HF 4.7 quick guide describes application communication on ports 8300 and 8301 by default.	Older guide; exact port roles and v4.9 behavior require runtime confirmation.
Verified	Strings show `waveIn*`, `waveOut*`, and `winmm.dll` including open, buffer, start/stop, and write operations.	Strong evidence that the Windows application references WinMM wave audio APIs. It motivates a portable Rust audio abstraction; it does not determine the replacement backend or complete audio format.
Verified	UI text includes separate input and output device names, plus AFC, `S/N @ 2500Hz`, baud-rate plot, and constellation diagram labels.	These are UI and diagnostic clues only. They do not reveal sample format, rates, modulation order, synchronization, coding, or demodulator design.

Class	Observation	Interpretation / limit
Static clue	Strings include `CQFRAME`, `LISTEN CQ`, `LISTEN ON/OFF`, `CHAT ON/OFF`, `RETRIES`, `VERSION`, `BITRATE`, and status/error text.	Candidate application-interface messages. Direction, syntax, response, ordering, and state semantics remain unverified.
Static clue	Ghidra cross-references map `CQFRAME`, `LISTEN ON/OFF`, and `VERSION` to `FUN\_004cdf80`; `CONNECTED`, `DISCONNECTED`, `BUSY ON/OFF`, `BITRATE`, and `VERSION VARA` reference separate functions `FUN\_004d14e0`, `FUN\_004d1ac0`, `FUN\_004d2640`, `FUN\_004d27a0`, `FUN\_004d2da0`, and `FUN\_004d33b0`.	These are static string xrefs in the VB6 application. The decompiler still cancels on these candidates; no command parsing, output, or state transition has been confirmed.
Static clue	ChangeLog entries mention KISS TCP frame types, fragmentation, DCD, persistence, speed levels 1/2/4, `BITRATE`, `DRIVELEVEL`, and special callsign suffixes.	Useful test-case leads; validate each against v4.9 behavior and protocol captures before encoding it in the implementation.
Static clue	Payload includes `VARAHF500.dat`, `VARAHF2300.dat`, and `VARAHF2750.dat`.	Distinct mode-named data files exist. Their data structures and role are unknown; no binary reverse-engineering conclusion is recorded yet.
Pending	Some sampled Ghidra decompilation requests returned `decompile_failed:cancelled`, including candidate functions near `004c9d90`, `004ca190`, `004cdf80`, and `004d2da0`.	Cross-reference and disassembly observations remain useful navigation evidence, but function semantics are not considered confirmed from those failed decompilations.

4.4 Saved Recon bookmarks

The analysis program has saved bookmarks at `00448260` (command-port UI text), `004cdf80` (references to CQ/listen text), `004d2da0` (BITRATE text), `00448a54` (sound-device readiness error), `00449164` (CQFRAME), `00440164` (WinMM audio API strings), `00442d1c` (audio input/output device labels), `004412a0` (audio diagnostics labels), `00449244` (RETRIES), and `00449508` (connection/status strings). These bookmarks are navigation aids, not proof that each referenced function implements the apparent behavior.

5 Rust scaffold and cross-platform environment

The crate is under `rust/`, package name `vara-modem`, edition 2024, Rust minimum 1.85. It currently has no third-party dependencies. Module boundaries are:

- `api`: future application-facing command/status interface;
- `audio`: future audio-device integration;
- `modem`: future portable modem/link behavior;
- `platform`: operating-system integration.

The executable prints help and version only. The crate deliberately does not claim modem functionality.

Rust 1.99.1 stable, Cargo, rustfmt, Clippy, and rust-analyzer are installed through rustup. Target standard libraries were installed for these six platform/architecture pairs:

```
aarch64-apple-darwin
x86_64-apple-darwin
aarch64-pc-windows-msvc
x86_64-pc-windows-msvc
aarch64-unknown-linux-gnu
x86_64-unknown-linux-gnu
```

`cargo check` passed for each target. This is cross-target Rust compile validation only; it is not native Windows/Linux execution, linking of platform dependencies, packaging, or audio-device validation.

Commands used for the Rust checks:

```
cd rust
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test
cargo check --target aarch64-apple-darwin
cargo check --target x86_64-apple-darwin
cargo check --target aarch64-pc-windows-msvc
cargo check --target x86_64-pc-windows-msvc
cargo check --target aarch64-unknown-linux-gnu
cargo check --target x86_64-unknown-linux-gnu
```

6 Workspace organization

Reference assets now live under `reference/vara/` and `reference/ghidra/`. The extracted v4.9 application payload is kept in `reference/vara/extracted-v4.9.0/app/`. Rust source and build instructions are under `rust/`; workspace overview is `README.md`. The live Ghidra project, VS Code MCP config, and bridge virtual environment remain at their existing paths. The current workspace is not a Git repository, so the notes do not identify a commit or claim source control tracking.

7 Implementation roadmap (proposed)

This roadmap is a plan, not a claim that the listed behavior has been reverse-engineered or implemented. Work is organized around measurable interoperability gates. In particular, waveform and decoder compatibility should be established before substantial investment in ARQ, host integration, or a graphical interface. The Rust crate now includes an early radio-control MCP prototype, but

no modem implementation; crate names below are logical boundaries and do not require creating every crate up front.

7.1 Planning targets and architecture

The proposed product is an independently implemented Rust modem that interoperates with an unmodified VARA HF installation. The existing project scope is macOS, Windows, and Linux on ARM64 and x86_64; native build, audio, and device validation are separate acceptance requirements for each supported platform. A modular Rust workspace should separate:

- applications and diagnostics (CLI, analyzer, and, later, GUI);
- host command/data interfaces and link/ARQ state;
- physical-layer synchronization, modulation, and waveform processing;
- coding, framing, and reusable DSP primitives; and
- platform adapters for audio, PTT, CAT, serial, and configuration.

Start with the existing crate and split components when stable interfaces and tests justify the boundary. Candidate libraries include `rustfft` and `num-complex` for analysis, `hound` for WAV fixtures, `cpal` for audio, `tokio` for asynchronous networking, `serde`/`toml` for configuration, and `tracing` for diagnostics. These are options to evaluate, not dependencies already selected. Keep the modem native Rust; Python may be used for optional research tooling.

The proposal's approximately 2.4 kHz bandwidth, 48 kHz audio, OFDM, eleven adaptive levels, FSK ACK signaling, turbo coding, and TCP compatibility are candidate requirements to investigate. None is established by the current evidence. Record measured values and their supporting fixtures before treating them as implementation requirements. SDR support is a possible later extension, not a first-release gate.

7.2 Phases and exit gates

The estimates below are rough engineering effort, not calendar commitments; parallelism, reference access, and undocumented behavior may change them. Each phase exit requires the stated evidence and regression tests, not merely compiling code.

Phase 0 — Research and reference environment (1–2 weeks). Establish authorized reference access, available specifications, a controlled audio loopback/capture setup, and a clean-room workflow. Record idle and representative non-radiating audio observations; maintain a corpus with provenance and known limitations. Deliver a research record, initial waveform corpus, and compatibility requirements with unknowns called out. Exit when a reference observation can be reproduced and its test conditions documented. Dynamic transmission work remains limited to controlled, non-radiating setups.

Phase 1 — Rust foundation and waveform analyzer (2–4 weeks). Retain the current scaffold while establishing shared error/logging conventions, WAV sample handling, spectral analysis, and machine-readable analysis output. Add visualization only where it helps inspect fixtures. Deliver an analyzer and deterministic tests over the captured corpus. Exit when it consistently detects and measures features demonstrated by the reference recordings; do not encode guessed waveform constants as facts.

Phase 2 — Synchronization and ACK receiver (2–4 weeks). Using evidence from Phase 0/1, characterize signal detection, frequency offset, timing, ACK signaling, symbol mapping, and session-dependent encoding. Implement a receiver only for behaviors supported by validated captures. Deliver repeatable ACK regression fixtures and command classifications where the evidence supports them. Exit when validated captures decode reliably across the documented cases; revise the phase if the proposed ACK model is incorrect.

Phase 3 — DATA physical-layer receiver (4–8 weeks). Characterize synchronization, carrier mapping, pilots, tracking, channel estimation, equalization, and demodulation from reference captures. Preserve soft information when required by the measured coding chain. Deliver a symbol-level receiver with per-capture diagnostics. Exit when it demodulates the validated reference cases across every confirmed transmission mode or level; the proposal's eleven-level count is not presumed.

Phase 4 — FEC and DATA decoding (4–8 weeks). Determine the actual encoder, interleaver, puncturing, frame-control, and integrity-check behavior before implementing it. Build encoder/decoder primitives and fixtures from independently observed inputs and outputs. Deliver complete frame decoding with payload and integrity validation. Exit when captured payloads are recovered and checks pass for the validated cases.

Phase 5 — Compatible transmitter (4–6 weeks). Implement the measured bit/frame construction, coding, modulation, pilots, and waveform generation. Add sample-level regression checks and validate against an unmodified reference receiver in a controlled, non-radiating setup. Exit only when the reference receiver accepts OpenVARA transmissions; visual similarity alone is insufficient.

Phase 6 — ARQ and session management (4–8 weeks). After physical-layer compatibility, implement the evidenced negotiation, roles, sequencing, ACK/NACK, retransmission, turnaround, adaptive behavior, timeouts, and PTT timing. Deliver a tested link state machine and transfer fixtures. Exit when bidirectional transfers complete reliably with an unmodified reference station under controlled test conditions.

Phase 7 — Host interface and platform integration (3–5 weeks). Confirm command and data socket semantics before implementing a compatible host interface. Add audio backends, configuration, and any evidenced CAT, serial PTT, or VOX integration. Deliver platform builds and host-application tests. Exit when supported host applications work without modification and the audio/PTT paths pass native validation on each claimed platform.

Phase 8 — Hardening and initial release (3–6 weeks). Profile and optimize only after correctness; add channel simulations, multipath/Doppler cases where applicable, extended interoperability tests, CI for formatting, linting, tests, and cross-platform builds, plus user and developer documentation. Review licensing and trademarks before distribution. The proposed 0.1 release gate is reproducible builds and demonstrated interoperability, not parity with VARA's maximum throughput or weak-signal performance.

7.3 Cross-cutting standards and acceptance

Use stable Rust and the existing edition/toolchain policy. Require formatting, Clippy, unit and integration tests, and cross-target compilation in CI; run native audio and interoperability tests

separately because compilation does not demonstrate device behavior. Prefer deterministic DSP and explicit error propagation. Avoid heap allocation in real-time callbacks where feasible, and require documented justification and review for unsafe code in DSP/protocol paths. Benchmark and optimize against measured workloads.

Release acceptance should include, at minimum: validated synchronization; decoded frames with integrity checks; reference acceptance of transmitted frames; reference decoding by the OpenVARA receiver; bidirectional ARQ transfers; verified host-interface behavior; correct measured turnaround and PTT timing; native platform/audio checks; and extended regression testing. Only claim the modes, platforms, and conditions actually tested.

7.4 Initial execution order

The first modem milestone remains a modest *Foundation 0.1*: add WAV fixture support and basic spectrum analysis, then establish CI for supported targets. A containerized radio-control MCP slice has been implemented in parallel as project infrastructure; it does not complete Foundation 0.1 or demonstrate modem compatibility. In parallel, unblock the authorized reference runtime and collect reproducible captures. Choose each receiver task from measured evidence. Defer the GUI, broad host compatibility, full ARQ, and speculative protocol implementation until their prerequisite gates are met.

7.5 Progress against the roadmap (2026-11-07)

The following status separates the recent radio-control implementation from the modem milestones and their evidence gates:

- **Phase 0 — partial; exit gate unmet.** Static reference analysis, extracted payload preservation, and research documentation are in place. The Windows guest has not provided usable runtime access; no controlled VARA audio captures or reproducible protocol observations have been collected.
- **Foundation 0.1 / Phase 1 — not started.** The Rust package builds, but WAV fixture handling, spectrum analysis, corpus-backed measurements, and CI for the supported target matrix are not yet implemented. The radio MCP service is not a waveform analyzer.
- **Phases 2–6 — not started; evidence-blocked.** No ACK/DATA receiver, FEC implementation, compatible transmitter, or ARQ/session state machine has been implemented. Their protocol assumptions remain unverified pending reference captures and confirmed command behavior.
- **Phase 7 — partial, radio-control slice only.** A Rust stdio MCP service runs in Compose with simulated and Hamlib 4.7.2 `rigctld` backends. Simulated frequency, mode, and power writes are process-local; physical CAT writes are disabled by default and require station limits. Capability enumeration parses one Hamlib `dump_caps` response into common CAT descriptors and generic levels, functions, and parameters. Image builds, Compose configuration, seven Rust unit tests, and MCP state/control/capability calls against the simulator and Hamlib dummy model have passed. No VARA host interface, audio backend, physical radio test, or native platform/device validation has passed.
- **Phase 8 — partial foundation only.** Rust formatting, compilation, the current unit tests, and container builds have been checked. Cross-platform CI, broad interoperability regressions,

performance qualification, and release/license review remain open.

This work verifies the MCP server boundary and Compose packaging; it is not evidence of VARA HF interoperability. The CAT catalog includes common frequency, mode, RF power, VFO, passband, split, antenna, memory-channel, RIT/XIT, and generic Hamlib level/function/parameter descriptors. Memory records, vendor extensions, complete non-control rig operations, and typed parameter enums remain open. PTT key-up remains denied pending a session/one-use-permit arbiter, and physical controls have not been tested. The local state revision does not yet detect changes made outside Linkwise. Keep hardware writes disabled until lease, permit, readback, audit, and watchdog behavior has been independently exercised.

8 Signal data acquisition and corpus (proposed)

8.1 Purpose and evidence policy

Build a traceable corpus of authentic VARA HF audio to support signal detection, synchronization, OFDM/ACK analysis, modulation and coding research, frame decoding, ARQ characterization, and interoperability tests. Compatibility with unmodified VARA HF is the target: generic OFDM, simulated signals, and VARA FM recordings are not substitutes for VARA HF reference waveforms.

Treat every conclusion as evidence with provenance. Use the labels `DOCUMENTED` (explicitly stated in a technical reference), `MEASURED` (measured from a recording), `VALIDATED` (confirmed by repeatable decoding or interoperability tests), `HYPOTHESIS` (plausible but unconfirmed), or `UNKNOWN` (insufficient evidence). Do not promote a hypothesis based only on visual resemblance or an isolated capture. Update roadmap assumptions when new evidence changes scope, ordering, or estimated complexity.

8.2 Candidate data sources

1. **Controlled virtual-audio sessions (preferred starting point).** The user-provided guide points to the N4EAC VARA Modem Tester, a Windows test harness described as connecting two installed VARA HF instances over virtual audio cables, exchanging payloads, and recording audio and session metadata. Its repository documentation should be checked before use. The guide lists Windows, legitimate VARA installations, independent virtual-audio routes and ports, Python 3.12, and sufficient storage as prerequisites. The harness does not supply VARA, virtual-audio drivers, or a reference corpus. It is a proposed capture tool, not evidence that every rate or frame type has been exercised.

2. **Actual HF receive recordings.** These add real-channel conditions not represented by clean virtual audio. Record radio receive audio and, where available and authorized, the modem-to-radio transmit audio path or cooperating station's capture. Begin receive-only or use a properly attenuated, controlled setup. Any over-the-air transmissions must follow the operator's licensing, authorization, and applicable operating procedures. Obtain permission before retaining or sharing recordings; do not publish private message contents without authorization.

3. **Public references and samples.** The Signal Identification Wiki VARA HF page is background and a pointer, not confirmation that downloadable raw audio is available. The VARA HF Specification Revision 2.0 may inform documented expectations, but the guide dates it to April 2018; treat it as a historical specification, not a complete description of current behavior. The

Freesound VARA FM recording may exercise WAV ingestion and analysis tooling only. It is VARA FM, not a VARA HF protocol fixture.

4. **Official VARA software.** The guide points to the Winlink VARA download page and downloads.winlink.org as sources for legitimate reference software. Software installation alone does not replace recorded, labeled signal data.

8.3 Recording preservation and metadata

Prefer complete, unprocessed WAV recordings: 48 kHz, mono where possible, uncompressed PCM at 16- or 24-bit depth, covering a complete transmission or session. These are capture preferences, not established VARA requirements. If the equipment produces another format or rate, preserve that original. Analysis copies may be converted, but must be traceable to the original. Never normalize, apply AGC, trim silence, denoise, filter, compress to MP3, or resample the only copy of a reference recording.

For every recording, retain a JSON sidecar and SHA-256 hash of the original. The sidecar should identify capture ID, mode, source type, file, audio properties, bandwidth when known, direction, DATA level when independently established, session, expected payload references, contact status, payload verification, SNR/frequency offset when measured, UTC capture time, source, license/permission status, and notes. Unknown values must be JSON `null`; never invent a DATA level, SNR, or frequency offset. A metadata example is a schema illustration only, not a record of acquired data.

```
{
  "capture_id": "vara_hf_bw2300_session_0001",
  "mode": "VARA_HF",
  "source_type": "controlled_virtual_audio",
  "file": "station_1.wav",
  "sample_rate": 48000,
  "channels": 1,
  "bit_depth": 16,
  "encoding": "PCM",
  "bandwidth": "BW2300",
  "direction": "station_1_transmit",
  "data_level": null,
  "session_id": "session_0001",
  "payload_file": "payload.bin",
  "expected_payload_file": "expected_payload.bin",
  "contact_status": "complete",
  "payload_verified": true,
  "snr_db": null,
  "frequency_offset_hz": null,
  "capture_date_utc": "YYYY-MM-DDTHH:MM:SSZ",
  "source": "N4EAC VARA Modem Tester",
  "license": "recording_owner_permission_required",
  "sha256": "computed hash of the original WAV",
  "notes": "Illustrative schema only; not an acquired capture"
}
```

8.4 Coverage and capture campaigns

Maintain a coverage report that distinguishes available, missing, and insufficiently validated cases. Initial categories from the guide are: connection handshake; START; ACK1/ACK2/ACK3; NACK; BREAK; REQ; QRT; DATA levels 1–4, 5–8, and 9–11; both directions; complete sessions; retransmission/failure; weak signal; frequency offset; and fading/multipath. The ACK names and eleven-level grouping are drawn from the supplied guide's published-specification summary and are not independently validated here. Bandwidth and data level are distinct: never infer a DATA level from BW500, BW2300, or BW2750, or assign a level without reliable metadata or independent signal analysis.

Use staged campaigns:

1. **Clean baseline:** use two VARA HF instances over separately routed virtual audio; begin at BW2300. Exchange a short known text payload (for example `OPENVARA TEST 001`), a longer repeatable text payload, and a known binary payload (the guide's example is `00 7F 80 FF 55 AA 00 FF`). Record both directions and compare transmitted and received payloads exactly. First establish one successful session before expanding the run.
2. **Bandwidth coverage:** repeat both directions at BW500, BW2300, and BW2750. A session's observed rate report is useful metadata, but does not establish a frame-level modulation assignment.
3. **Adaptive rates:** vary channel conditions where possible and seek low-, intermediate-, and high-rate examples, rate increases and decreases, and retransmissions. Do not assume one session covers all eleven published levels; verify each assignment independently.
4. **Real channel:** for permitted HF captures, note frequency/band, UTC time, receiving-station configuration, signal strength or SNR if available, interference, frequency offset if measurable, and decode outcome. Obtain permission and protect private content.
5. **Failure and recovery:** collect poor synchronization, weak signals, failed DATA, NACK/retransmission, failed-ACK recovery, termination, and interrupted sessions under controlled, authorized conditions.

8.5 Storage, analysis, and corpus admission

The proposed repository layout is a dedicated `data/` tree with `public/vara_hf` and `public/vara_fm`, `generated/clean`, `generated/bandwidth`, `generated/data_levels`, `generated/acknowledgments`, `generated/sessions`, field-capture folders for clean/fading/interference/weak-signal cases, `derived/` folders for plots and measurements, and `reports/` for coverage and analyses. This is a proposal, not a statement that these directories currently exist. Keep large or restricted files outside the public source tree where appropriate; retain a manifest and approved-storage references. Do not commit recordings without permission and redistribution rights.

For each candidate recording, validate the WAV header and calculate duration, RMS, peak, and clipping percentage. Produce waveform and spectrogram views; measure occupied bandwidth and burst start/end, and estimate symbol timing, cyclic-prefix behavior, carrier spacing, frequency offset, and candidate pilots only when the data supports those measurements. Compare with published references, record confidence and evidence, and save a machine-readable analysis report. Map results to implementation areas: waveform/spectrum to DSP; burst detection and symbol

timing to synchronization; carrier mapping to OFDM; ACK tones to FSK; modulation mapping to codec; soft decisions to FEC; DATA/ACK framing to link; retransmission/changeover to ARQ; and end-to-end traces to link and ARQ.

A recording enters the validated reference corpus only when it is readable, the original is preserved, format and sample properties are identified, actual modem activity is supported by evidence, source/capture method and permission status are documented, a sidecar and hash exist, category assignment is justified, and any expected payload has been independently verified. Quarantine incomplete or uncertain files; do not use them as golden fixtures. Existence of a WAV file is not proof of a successful capture.

8.6 Initial acquisition sprint and gate

The proposed first sprint favors a small, well-labeled corpus over many unverified files. Target one clean session each at BW2300, BW500, and BW2750; three station-1 and three station-2 WAV recordings; three known text payloads and one known binary payload; a sidecar for every recording; a complete SHA-256 manifest; waveform plots and spectrograms for every recording; and one coverage report. These are targets, not current assets or completed measurements.

This corpus can start WAV ingestion, waveform analysis, signal detection, and early synchronization work; it is not sufficient to claim full compatibility. The immediate data milestone is a reproducible, validated collection of original VARA HF WAVs with known payloads, metadata, hashes, and analysis reports. Do not implement the complete modem solely from a published specification. The recommended order is to establish the capture harness and corpus/metadata first, analyze every accepted recording with repeatable tools, and then add permitted real-channel captures. Reassess coverage because a capture tool is not proof that all rate levels or control frames were tested.

9 Roadmap unblock plan

9.1 Primary blocker and objective

The critical path is access to a working, authorized VARA HF runtime and repeatable audio capture. Until an authentic reference session is observed, the project cannot ground waveform measurements, frame decoding, ARQ behavior, or end-to-end compatibility claims. Static analysis and the 2018 specification can guide questions, but they do not clear this gate. The immediate objective is deliberately narrow: obtain one controlled, bidirectional VARA HF session with known payloads, original audio from both directions, and enough metadata to reproduce and verify the result.

9.2 Ordered recovery and decision gates

1. **Triage the existing Windows guest, without destructive changes.** Start the reported Windows 11 ARM64 VM in UTM and establish its actual state: boot to desktop, guest sign-in, VARA launch, and guest network visibility. Record any exact startup error and inspect UTM's VM status/logs before changing configuration. Confirm the installed VARA version and that the required authorized installation is usable. *Gate:* a working desktop with VARA running and a repeatable way to reach its local command interface. If the guest fails, make one bounded diagnosis based on the observed failure rather than repeatedly changing unrelated VM settings.

2. **Use an alternate legitimate Windows runtime if VM recovery stalls.** The preferred fallback is an already available, authorized Windows computer or VM with VARA installed, plus controlled virtual audio routing. Confirm it can run the reference application before spending more effort on VM infrastructure. The Linux/Wine guest is a secondary fallback only; record its results as Wine-specific and do not treat them as Windows compatibility evidence. Public samples or VARA FM audio may test file-analysis tooling, but cannot clear the VARA HF runtime gate.

3. **Prepare the capture path and corpus bookkeeping in parallel.** Follow the signal-data guide: establish two independent audio routes for the two endpoints, configure capture of both directions, confirm ports and endpoints, and create the manifest/sidecar location before running a session. Use only a controlled non-radiating audio path. Confirm that original recordings can be written and read back and calculate hashes; do not overwrite originals during analysis. *Gate:* a short test capture is readable, non-silent when expected, and has the correct audio headers and routing. A route check is not itself a VARA protocol capture.

4. **Run the minimum end-to-end reference session.** Start at a single bandwidth setting (the guide suggests BW2300), exchange a short known text payload and a known binary payload, record both stations' audio in both directions, and preserve the application transcript and exact sent/received bytes. Confirm the session completed and payloads compare byte-for-byte before accepting the WAVs. Do not begin with rate sweeps or field transmissions. *Gate:* at least one reproducible session, complete original audio from each direction, verified payloads, UTC capture time, source and setup notes, JSON sidecars, and SHA-256 hashes.

5. **Validate and index the first corpus entry.** Check file readability, duration, sample properties, modem activity, metadata, permission status, and payload verification. Label conclusions with the evidence taxonomy. Quarantine incomplete or uncertain files. *Gate:* the capture meets every corpus-admission criterion in the signal-data section. Only then use it as a golden fixture or claim Phase 0 complete.

6. **Expand coverage only after the baseline works.** Repeat the controlled session at BW500 and BW2750, then target control frames, adaptation, retransmission, and failure cases with explicit coverage tracking. Add real-channel receive-only data only with appropriate permission and operating compliance. Do not equate bandwidth with DATA level or claim all levels from a session-level rate report.

7. **Continue the independent foundation track.** While runtime access is being restored, progress Foundation 0.1: WAV ingestion, header/sample validation, basic waveform/spectrum analysis, synthetic test fixtures for generic signal-processing correctness, CI, and cross-target checks. Label synthetic fixtures as synthetic; do not use them to infer VARA parameters. This keeps productive work moving without pretending the evidence gate has passed.

9.3 Stop conditions and escalation

Set a bounded first attempt: one focused Windows VM triage; if it does not yield a usable runtime, switch to the alternate authorized Windows environment instead of letting VM installation consume the protocol schedule. If neither Windows path is available, keep corpus-dependent protocol/DSP tasks explicitly blocked and continue only infrastructure work that does not encode assumed VARA behavior. Do not substitute downloaded, uncertain, private, or incompatible-mode recordings for authentic VARA HF evidence. Do not transmit over the air as a workaround; use receive-only

captures or a controlled, non-radiating setup.

The unblock plan succeeds when a clean reference session has been reproduced, both directions and payloads independently verified, and its original files admitted to the corpus with provenance, permission status, metadata, hashes, and analysis report. This unlocks evidence-led waveform analysis; it does not by itself establish full modem compatibility or complete Phase 0 coverage.

10 Readiness snapshot: 2026-11-07 (historical)

This dated snapshot is retained as history. Section 11 records the later modem implementation and supersedes these readiness claims.

Ready / observed	Still pending
Ghidra 12.1.4 plus matching extension; live plugin health endpoint; actual VARA v4.9 executable imported; initial Recon bookmarks; extracted reference payload preserved; containerized Rust radio MCP prototype; simulated MCP control and Hamlib dummy-backend smoke checks; seven Rust unit tests; both container images built; six target checks; UTM and CrystalFetch installed; Windows installer ISO downloaded and located; UTM MCP configured; the user reports the `Windows` VM installation and ARM64 guest are configured; both UTM guests are currently stopped; the Windows guest has not provided usable network/runtime access; user states written authorization covers the analysis scope; UTM share `~/Downloads/utm-share/` is registered and contains the extracted v4.9.0 payload; command-related strings and additional function xrefs have been mapped in Ghidra.	Fedora install/first boot; guest network/runtime visibility; confirmed command behavior and controlled TCP/audio captures; confirmed command protocol; confirmed DSP/on-air behavior; native OS/audio tests; any working Rust modem features. The Windows ISO is present in Downloads.

11 2026-11-09: Native modem implementation and verification

This entry supersedes the 7 November readiness snapshot for modem work. The user requested completion of the remaining implementation stages and later confirmed that no usable VARA reference instance was configured. Native components and test tooling were implemented, but the stages below are not all complete. Local round trips and two-instance tests are not reference-peer interoperability evidence.

11.1 Implementation changes

- **Recorded-audio validation.** Added `tools/test-modem.py`. It downloads five published BW500 level-1/2 WAV excerpts and their provenance at a pinned upstream revision, checks SHA-256 values before use, and runs the otherwise ignored reference-replay test with the Rust suite. The excerpts include two frames in one burst. They are cached under the ignored `reference/vara/fixtures/` tree, not redistributed in source.
- **Streaming receiver and audio input.** Added `rust/src/modem/receiver.rs` with absolute sample positions, chunk-independent processing, non-finite input rejection, and at most 30 seconds of retained audio. Truncated candidates emit no partial payload. WAV replay uses this worker instead of retaining the entire recording. The optional `audio-device` feature adds CPAL capture at native 48 kHz, explicit device/channel selection, sample conversion, and a bounded queue. Capture fails on overflow or device error rather than joining discontinuous samples. Device enumeration worked on the ARM64 Mac; live device decoding was not qualified.
- **Level-4 waveform.** Added `rust/src/modem/bw500_waveform.rs`: two-band differential BPSK synthesis, fixed-pilot timing acquisition, matched-pulse demodulation, soft decisions, and CRC-checked decoding through the existing level-4 turbo codec. Single-frame nominal-rate audio is covered. Randomized acquisition leads, multi-frame behavior, carrier tracking, and sample-clock tracking remain open.
- **External measurements.** Added `tools/fetch-modem-profile.py` to fetch checksum-pinned published phase/pulse measurements and extract numeric literals with Python AST parsing, without executing downloaded code. The generated level-4 profile and attributed upstream sources remain in the ignored cache. The source identifies the measurement data as AGPL-3.0-only; no measurement tables are embedded in the Rust source.
- **Audio-driven handshake.** Added `rust/src/modem/audio_handshake.rs`, phase-specific expected bursts, the BW500 confirm generator, and complete caller-keyed connected-ACK matching. Two local instances establish only after audio confirmation and explicit transmit completion. Returning audio does not imply playback, PTT release, or reference acceptance.
- **DATA and ARQ components.** Added `rust/src/modem/data.rs` for uncompressed full/short containers, caller-keyed trailers, and CRC validation. The documented 200-byte delivery layout reconstructs without padding. The changing full-frame field is caller-supplied because its wire sequencing is not yet qualified. Added `rust/src/modem/arq.rs`: a bounded stop-and-wait sender, missing-answer queries, explicit negative-answer retries, query/retry budgets, transmit watchdogs, and retention of unresolved bytes on failure. These are event-level components, not an operational wire ARQ/session implementation.
- **Host boundary and bench CLI.** Implemented bounded, CR-delimited BW500/OFF command parsing in `rust/src/api.rs` and command/notification plus raw binary DATA stream transports in `rust/src/host.rs`. Tests cover fragmentation, coalescing, invalid-command recovery, binary preservation, and EOF cleanup. An operational modem backend and command/data socket ownership remain to be integrated. Added `render-wav` for file-only handshake stimuli and `receive-audio` for bounded device-input runs. Updated the implementation guide and modem status document.

11.2 Evidence provenance and reproducible checks

The external technical basis is Saul St John's published *hfmodem* VARA protocol notes, reference excerpts, and measured phase/pulse data, pinned at revision `952f79245240132f4af209afbad1f4b2ba256613`. The protocol documentation is published as CC-BY-4.0; the downloaded measurement sources identify their separate AGPL-3.0-only license. Original vendor binaries and private captures were not copied into the tracked source tree. The five WAV files retain upstream provenance, including source hashes and expected payloads. This session replayed published captures; it did not collect a new successful stock-VARA session.

From the repository root:

```
python3 tools/test-modem.py --audio-device
```

From `rust/`:

```
cargo fmt --check
cargo clippy --locked --all-targets --all-features -- -D warnings
```

Verified: the final recorded run passed **41 tests**: 37 library unit tests, one integration test covering all five reference recordings, and three level-4/audio integration tests. No tests were skipped in that runner. Formatting, Clippy with warnings denied, and the whitespace diff check passed. The ordinary offline `cargo test` still reports external-data tests as ignored unless their prerequisites are provided.

The level-4 integration tests cover a setup-frame audio round trip, a complete local two-role audio handshake, streaming DATA reception, and rejection of heavily corrupted audio. A CLI smoke check also rendered a W9SSJ setup stimulus as 205,825 samples of 48 kHz mono float WAV and decoded one CRC-valid level-4 frame with control byte `0x82`. These level-4 tests are synthetic and use the external measured profile. The independently recorded peer audio coverage remains levels 1 and 2; level 3 has synthetic round-trip coverage.

11.3 Attempted reference runtime: failed environment gate

Docker Desktop was started and a temporary isolated 32-bit bench built on the ARM64 Mac using i386 Debian bookworm, Wine 8, Xvfb, and PulseAudio. The existing VARA 4.9.0 extraction was mounted read-only and copied into the bench's writable working directory. The Microsoft VB6 SP6 runtime was obtained through the archive URL used by Winetricks 20250102 and checked against SHA-256:

`467b5a10c369865f2021d379fc0933cb382146b702bbca4bcb703fc86f4322bb`.

Wine initialized, but repeated COM registration attempts under qemu-i386 failed with `Unhandled illegal instruction at address 7BC57F86`; WineDbg then reported error 5 while attaching. The program-error dialog was visually verified. No functioning VARA control endpoint or reference audio loopback resulted. The experimental container was stopped after diagnosis. This is an *environment failure*; it supplies no acceptance or rejection result for the native modem's transmitted frames.

11.4 Stage status and unresolved gates

- **Waveform: partial.** Native single-frame level-4 synthesis and reception pass local tests. Independent stock recordings, acquisition leads, multi-frame operation, and impairment tolerance remain open.
- **Handshake: partial.** Two local instances establish through decoded audio. Device playback, turnaround, PTT ownership/release, and live reference acceptance remain open.
- **DATA/ARQ: partial.** Container codecs and the bounded sender exist. Wire field sequencing, reply/query codecs, duplicate handling, rate changes, turns, keepalive, and disconnect still need integration and qualification. A missing answer must not authorize blind DATA retransmission.
- **Host interface: partial.** Command and DATA stream components pass tests. They are not connected to a complete operational modem service, nor to Winlink message handling.
- **Qualification: exit gate unmet.** No bidirectional transfer with an unmodified VARA peer, live audio/PTT qualification, or native cross-platform device qualification was completed.

The maintained implementation inventory is `rust/docs/modem.md`. Its stage table must remain explicit about these limits; 41 passing tests do not establish a complete interoperable modem.

11.5 Whitepaper build and overflow fixes

Used `tools/build-whitepaper.sh` to rebuild the architecture whitepaper and all PlantUML vector figures. The initial build succeeded with horizontal overflow warnings. Widened the requirements table's method column, made file paths breakable, allowed modest emergency line stretch, reworded one unbreakable reference phrase, and reduced the requirements table font so its final row no longer occupied a separate landscape page. The rebuilt paper had no overfull-box warnings; affected pages were rendered and visually inspected. This layout/build result is separate from modem implementation evidence.

12 2026-11-10: Shared runtime and BW500 audio loopback

12.1 Architecture and implementation

Updated the architecture whitepaper, sections 14.1–14.4, to define the shared Rust runtime, standalone and managed traditional-client admission, session ownership and device completion contracts, and staged exit criteria. Updated the session-start, runtime, keying and ARQ recovery diagrams. Rebuilt the paper and inspected the changed pages and diagrams.

Implemented the shared session owner in `rust/src/modem/runtime.rs`. It integrates the existing establishment and ARQ state machines at the validated protocol-event boundary. It bounds byte and event queues, tracks pending, outstanding, acknowledged, unresolved and discarded bytes, and preserves unresolved payload on ambiguous failure. Matching presentation and observed release are required before the protocol records a completed transmission. Capture gaps and changed epochs fault the session.

Added the Tokio runner in `rust/src/modem/runtime/runner.rs`. It polls the device and advances deadlines on a monotonic clock at 5 ms intervals. Device observations and aborts use a separate bounded priority queue. Dropping the last handle faults a live session and requests release while

retaining unresolved payload. The join result reports whether release evidence arrived. Paused-clock tests covered watchdog expiry while a client was idle, abort ahead of a full normal command queue, and release after the last handle closed.

12.2 Simulated audio experiment

Added the audio loopback in `rust/src/modem/runtime/audio_loopback.rs`. The foreground command uses `-simulate-audio`. Two Rust runtime instances completed the level-4 establishment handshake and exchanged CRC-protected frames through a simulated half-duplex audio channel at 48 kHz. The cable model applied attenuation, sample offset and turnaround. The bench advanced virtual time by the sample duration and compared both delivered streams byte for byte.

- **Caller to responder.** Delivered 79 bytes in three DATA attempts. Erasing the first acknowledgment caused a state query. The receiver's positive reply retired the block without retransmission.
- **Responder to caller.** Delivered 70 bytes in four DATA attempts. Erasing the first DATA waveform caused a state query. Its negative reply authorized one retransmission.
- **Duplicate delivery.** Replaying one decoded DATA record per direction caused no additional delivered bytes.

The bench uses the existing BW500 establishment and level-4 waveform codecs with a private payload format for sequence numbers, lengths, queries and answers. The waveform profile was generated from the pinned, checksum-verified measurements used by the 9 November tests. The published BW500 frame notes describe role-dependent replies. The published stock recovery evidence covers wider bandwidths. The bench does not establish production BW500 DATA, reply, query or turn behavior against an independent peer.

12.3 Reproduction and results

Run from the repository root with Rust available on `PATH`.

```
python3 tools/test-modem.py
cargo run --manifest-path rust/Cargo.toml --locked --bin linkwise-modemd -- --simulate
profile=$(python3 tools/fetch-modem-profile.py)
cargo run --manifest-path rust/Cargo.toml --locked --bin linkwise-modemd -- --simulate-
    audio "$profile"
cargo fmt --manifest-path rust/Cargo.toml -- --check
cargo clippy --manifest-path rust/Cargo.toml --locked --all-targets -- -D warnings
```

Verified on macOS ARM64 with Rust 1.88.0. The fixture-enabled suite passed 60 tests with zero failures or skips. The clock-driven protocol simulation acknowledged 256 bytes after one DATA attempt and a state query. The audio bench reported 79 and 70 exact bytes, three and four DATA attempts, two queries and two suppressed duplicates. Formatting and Clippy passed. The earlier shared-core build also passed compile checks for x86-64 Linux and Windows. Those checks did not exercise native device access.

Pending Independent BW500 DATA sequencing, reply/query recovery and turn qualification. Native audio/PTT execution, paired TCP host access and a transfer with an unmodified VARA peer remain separate gates. The audio loopback used two Linkwise instances, simulated devices and virtual time.

13 2026-11-10: KiwiSDR receive evidence and pilot acquisition

13.1 Host and platform work

Added a paired loopback TCP bench with a simulated peer. The macOS socket test transferred 1024 binary bytes across both attachment orders, rejected duplicate connections, and verified cleanup on loss of either channel. Added a bounded 48 kHz output primitive with a CPAL presentation estimate. macOS enumerated output devices. The audio feature cross-checked against Linux ALSA metadata and Windows MSVC, in that order. Hardware playback timing and PTT release remain unqualified. Three stock-derived BW500 intermediate reply vectors matched the new encoder.

13.2 Live capture and first verified frame

The initial survey collected 17.5 receiver-minutes in 11 USB and IQ captures on 80, 40, and 20 meters. The follow-up recorded additional IQ in North America and Chile. Attempts in the UK, Australia, and New Zealand yielded no usable samples. The capture manifests record receivers, tuning, sample rates, UTC windows, sequence continuity, and source hashes.

The first automatic replay recovered no CRC-valid frames. Noise kept six unscaled USB captures above the fixed 1% RMS threshold long enough to exceed the 30-second candidate history. In a controlled replay, both stock-derived level-1 and level-2 frames mixed with Kiwi receiver noise decoded at +12 dB. At +6 dB the detector split the two signals into 27 and 28 candidates. Supplying the intact boundaries recovered both frames byte for byte.

Pilot matching then located a frame in the existing Lake Merwin, Washington KiwiSDR capture on 7.105 MHz USB. Capture began at about 5:25 p.m. PST on 10 November (01:25 UTC on 11 November). The recorded IQ center was 7.1065 MHz, with a WAV header rate of 11999 Hz. The pilot score was 0.758. The decoder recovered one CRC-valid BW500 level-2 frame with a 23-byte wire body, including control byte 0. Six replay filter widths from 290 to 420 Hz recovered the same body from that one frame. Its SHA-256 is `d32af06df32522e169366eedc5bb5e818724265a84ef310466c5fb728682b212`.

A separate checksum-verified published off-air VarAC exchange yielded three CRC-valid frames in the documented level order 2, 3, 2 when supplied with pilot-located windows. The third window used an 11.71875 Hz frequency correction.

13.3 Acquisition change and verification

Implemented a bounded rolling pilot search for BW500 levels 1–3 in `rust/src/modem/bw500/scan.rs`. The receive commands score pilot patterns every 128 samples against energy in the occupied band and guard bins, test 0 and ±12 Hz, refine timing, and pass acquired frames to the existing CRC-gated decoder. Overlap suppression prevents repeated emission of the same frame. Level 4 retains energy-based acquisition. The WAV report now counts truncated energy candidates while returning independently recovered pilot frames.

- **Full Lake Merwin recording.** Recovered the same live frame once with both 330 Hz and 1200 Hz IQ low-pass widths. Replay used the full demodulated recording without supplied frame boundaries.
- **Published off-air exchange.** Recovered the three frames in order with matching body hashes and no supplied boundaries or per-frame correction.

- **Kiwi noise controls.** Automatically recovered both +6 dB test frames byte for byte. The Maui receiver noise recording emitted zero frames.

The fixture-enabled suite passed 68 tests on macOS with the audio feature enabled. Formatting and Clippy passed. The new regression exercised consecutive frames under continuous interference and checked chunk independence and duplicate suppression.

```
python3 tools/test-modem.py --audio-device
cargo clippy --locked --manifest-path rust/Cargo.toml --all-targets --features audio-
    device -- -D warnings
```

Local evidence directories:

```
reference/vara/fixtures/kiwi-live-2026-11-11
reference/vara/fixtures/kiwi-vara-followup-2026-11-11
```

The follow-up directory contains the live verification, published exchange, and automatic acquisition results as JSON. The evidence establishes receive-side decoding of one live Kiwi frame and three published off-air frames. Live frame error rate, sample-clock tracking, production link behavior, and two-way interoperability remain open.

14 Chronicle maintenance

Update this log during the work, not only when milestones are complete. Add dated entries when the analysis scope changes, new evidence is collected, code or tooling is changed, a validation command is run, or a blocker is discovered or cleared. Label deductions as hypotheses until independently verified. Refresh the presentation when a milestone, demo capability, or material readiness claim changes. Keep generated PDFs rebuilt from the LaTeX sources so the presentation and notes do not diverge.

15 Next work

1. Establish a working unmodified VARA reference runtime. The user now reports no configured reference instance; the isolated Wine attempt failed its environment gate. Capture exact command/status bytes and both audio directions with provenance before making interop claims.
2. Qualify level-4 setup/DATA audio, acquisition leads, multi-frame behavior, and timing against independent recordings and the reference peer. Extend impairment and native device tests without promoting synthetic results to interoperability evidence.
3. Resolve and document wire DATA-field sequencing and recovery replies; integrate the container codec and bounded sender with rate, turn, keepalive, duplicate, and disconnect behavior. Preserve unresolved bytes and query after a missing answer rather than blindly repeat DATA.
4. Connect the host command/data transports to one operational session owner with explicit socket pairing, buffering, backpressure, and channel-loss cleanup. Parsing a command is not session acceptance.
5. Integrate device playback and measured turnaround with exclusive radio/audio ownership, one-use transmit permits, watchdog release, readback, audit, and the independent emergency stop. Keep physical writes disabled until the safety and qualification gates pass.

6. Complete controlled bidirectional reference transfers, impaired-link regressions, and native platform/audio qualification before claiming an operational cross-platform modem or Winlink compatibility.

16 References

1. Supplied Ghidra beginner guide: local guide. Relevant subjects include program import, decompiler, function graphs, and cross-references.
2. VARA HF v4.9.0 local installer and extracted ChangeLog: `reference/vara/installers/` and `reference/vara/extracted-v4.9.0/app/ChangeLog.txt`.
3. VARA product overview: https://varamodem.com/.
4. UTM Windows guest guide: https://docs.getutm.app/guides/windows/.
5. Microsoft Windows 11 ARM64 download: https://www.microsoft.com/software-download/windows11arm64.
6. Wing, R., Gerding, M., Plane, J. M. C., et al., "Measurement of a lithium plume from the uncontrolled re-entry of a Falcon 9 rocket," *Communications Earth & Environment* 7, 161 (2026), published 19 February 2026, https://doi.org/10.1038/s43247-025-03154-8.
7. Linkwise project, "HF propagation toolbox," research summary and experiment implications: `reference/propagation-toolbox.md`.
8. Saul St John (W9SSJ), *hfmodem*, published VARA protocol notes, reference recordings, and measured waveform constants, revision identified in Section 11: https://github.com/sstjohn/hfmodem.
9. Winetricks 20250102, VB6 runtime download recipe and verification hash: https://github.com/Winetricks/winetricks/blob/20250102/src/winetricks.